

Guía de estudio — Laboratorio 4

2026

1. Cómo funciona la evaluación

Van a ser preguntas múltiple opción con 5 opciones y una sola correcta.

Estrategia: estudiar **todas** las categorías, no solo el código. En la defensa pueden caer preguntas del enunciado, de Wikipedia, de Tanenbaum o de conceptos y métricas del paper .

Antes de la defensa, repase:

1. Su **paper** y **poster** (cifras reales del grupo, no los placeholders de la plantilla).
 2. El **kickstarter** ([Net.cc](#), [App.cc](#), [Lnk.cc](#), [network.ned](#), [omnetpp.ini](#), [packet.msg](#)).
 3. Lo que **cambió** su **Net** respecto al baseline.
 4. Resultados de **Caso 1**, **Caso 2** y **comparación** con el baseline.
 5. El enunciado y la bibliografía.
-

2. Plan de estudio sugerido (7 bloques)

Distribuya el tiempo en proporción al contenido para estudiar. Una semana tipo:

Día	Bloque	Tiempo sugerido	Archivos / lecturas
1	Enunciado y entrega	2 h	2026 Lab4_enunciado.pdf , esta guía
2	Kickstarter OMNeT++	3 h	lab4-kickstarter / *, trazar a mano como viaja un paquete por ejemplo del nodo 0 → al nodo 5

Día	Bloque	Tiempo sugerido	Archivos / lecturas
3	Anillo + dúplex	2 h	Red en anillo , Dúplex
4	Capa de red (Tanenbaum)	3 h	Cap. 5 (capa de red) del libro de redes
5	OMNeT++	2 h	Manual / tutorial OMNeT++, referido al lab4 y las simulaciones en base al código fuente puede buscar cada función que no sepa como funciona en el manual
6	Paper y poster + repaso	2 h	lab4-paper.tex , lab4-poster.tex , sus PDFs finales
7	Simulacro	1–2 h	genere 20 preguntas basadas en toda esta documentación para practicar intercámbielas con sus compañeros y traten de resolverlas

3. Bloque A — Enunciado y entrega

Qué debe saber

- **Objetivos del lab:** modelar en OMNeT++, analizar enrutamiento, diseñar estrategia mejor, redactar paper (6–8 páginas) y defender **poster** en la muestra presencial.
- **Modelo:** 8 nodos, **2 interfaces** por nodo, enlaces **full-duplex** entre vecinos.
- **Capas por nodo:** **app** (tráfico), **net** (enrutamiento), **lnk[0]** y **lnk[1]** (colas/enlace).
- **Rol nuevo:** **net** decide **por qué interfaz** reenviar (múltiples entradas/salidas).
- **Baseline de la cátedra:** si el paquete no es para este nodo → siempre **send(msg, "toLnk\$o", 0)** (sentido **horario**).
- **Caso 1:** **node[0]** y **node[2]** → **node[5]** (config por defecto en **omnetpp.ini**).

- **Caso 2:** nodos **0, 1, 2, 3, 4, 6 y 7** → `node[5]`; mismos `packetByteSize` e `interArrivalTime`; hallar umbral de **estabilidad** variando `interArrivalTime`.
- **Métricas pedidas:** demora E2E, **saltos**, utilización de **buffers** y **enlaces**, otras que agreguen.
- **Tarea de diseño:** algoritmo que **supere** al kickstarter; comparar Casos 1 y 2; discutir **bucles** de enrutamiento.
- **Tarea estrella (opcional):** topología **general**, **N interfaces**, gates **desconectados**; probar en `networkStar` (comentada en `network.ned`).
- **Restricción importante:** **NO** usar `cTopology` ni iteradores de `cGate` para descubrir vecinos → sí **paquetes echo/hello** y tablas locales.
- **Entregables:** código comentado, paper LaTeX, poster, defensa oral con poster.

Ficha rápida (memorizar)

None

Anillo: 8 nodos, 2 interfaces, full-duplex

Baseline: destino local → toApp; si no → toLnk[0] (horario)

Caso 1: emisores 0 y 2 → destino 5

Caso 2: emisores 0,1,2,3,4,6,7 → destino 5 (5 no emite)

Paper: 6-8 páginas | Poster: muestra + defensa

Prohibido: `cTopology` / `cGate` para vecinos

Errores típicos en las alternativas incorrectas

- Confundir Caso 1 con Caso 2 (quién emite).
- Creer que el baseline elige la interfaz con menor carga o usa `cTopology`.
- Pensar que `node[5]` genera tráfico en el Caso 2.
- Olvidar que echo/hello **sí** está permitido.

4. Bloque B — Kickstarter OMNeT++

Qué debe saber (archivo por archivo)

Net.cc

- Compara `pkt->getDestination()` con `getParentModule()->getIndex()`.
- Local → `send(msg, "toApp$o")`.
- Remoto → `send(msg, "toLnk$o", 0)` **siempre** (pregunta clave del lab).

App.cc

- Si `interArrivalTime == 0` → **no** programa `sendMsgEvent` (nodo sin tráfico).

- Generación: `scheduleAt(departureTime, sendMsgEvent)` con `departureTime = simTime() + interArrivalTime`.
- Crea `Packet` con `byteLength, source, destination`.
- Al recibir paquete: `demora = simTime() - getCreationTime() → delayStats / delayVector`.
- `finish(): recordScalar("Average delay", ...)` y "Number of packets".

Lnk.cc

- Cola `cQueue buffer`; servicio con `endServiceEvent`.
- Llegada por `toNet$i` → `insert` en buffer; si servidor idle → `scheduleAt(simTime()+0, endServiceEvent)`.
- Al terminar servicio: `pop, send(pkt, "toOut$o")`, próximo servicio en `simTime() + pkt->getDuration()`.
- Llegada por `toOut` (vecino) → `send(msg, "toNet$o")` hacia `net`.
- `bufferSizeVector` registra longitud de cola.

packet.msg

- Campos: `source, destination, hopCount` (deben **incrementarlo** en su solución si miden saltos).

network.ned

- `Network: node[8], interfaces=2`.
- Canales: `datarate = 1Mbps, delay = 100us`.
- Anillo: `node[i].toNod[1] <--> node[i+1].toNod[0]`; cierre `node[7]–node[0]`.
- `NetworkStar`: 57 nodos, 4 interfaces (comentada).
- `allowunconnected`: gates `toNod` pueden quedar sin cable (tarea estrella).

omnetpp.ini (Caso 1 por defecto)

- `sim-time-limit = 200s`
- `node[0]` y `node[2]`: `interArrivalTime = exponential(1), packetByteSize = 125000, destination = 5`

Ejercicio práctico

Trace **un paquete** de `node[0]` a `node[5]` con el baseline: liste cada módulo (`App → Net → Lnk → canal → ...`) y cuente **saltos** y **interfaces**. Compare con la ruta si hubiera usado `toLnk[1]`.

Errores típicos

- Confundir `toLnk[1]` con el comportamiento del kickstarter.

- Creer que **Lnk** descarta paquetes de **toOut** o que usa **cTopology**.
- Mezclar unidades: **100us** es delay del **canal**, **200s** es tiempo de **simulación**.

5. Bloque C — Topología en anillo

Basado en [Red en anillo \(Wikipedia\)](#) y en la relación con el lab.

Qué debe saber

- Cada nodo se conecta a **exactamente dos vecinos**; forma un **camino cerrado**.
- En anillo **unidireccional**, un fallo de enlace/nodo puede **partir** la red.
- **Doble anillo** (contrarrotación): redundancia ante fallos (FDDI, etc.).
- **Token Ring (802.5)**: a menudo **estrella física** + MAU; el “anillo” es lógico en capa 2 — no confundir con el anillo del lab.
- **Ancho de banda** en anillo compartido: se **comparte** entre nodos.
- **Demora** suele crecer con la **cantidad de saltos** en el camino.
- Con **2 interfaces**, un algoritmo inteligente puede elegir sentido **horario o antihorario**; el baseline no.
- En 8 nodos, sentido único: hasta **7 saltos** entre extremos opuestos.
- **Caso 2**: muchas fuentes al mismo destino → riesgo de **colas** y demora (inestabilidad).
- **Estabilidad**: colas acotadas; explorar variando **interArrivalTime**.
- **networkStar**: probar el algoritmo en topología **distinta** (más nodos, más grado).

Conectar con su trabajo

Concepto teórico	En su simulación
Camino único (baseline)	Solo toLnk[0] → una ruta 0→1→2→3→4→5
Camino más corto	Su algoritmo podría usar interfaz 1 si el destino está “del otro lado”
Congestión	Buffers de Lnk crecen si la oferta supera 1 Mbps × tramos
Partición	Rotura de enlace (no modelada por defecto) aislaría nodos

6. Bloque D — Dúplex y medios

Basado en [Dúplex \(telecomunicaciones\)](#).

Qué debe saber

Modo	Idea clave
Full duplex	Transmitir y recibir a la vez (enlace punto a punto dedicado)
Half duplex	Alterna sentidos; no simultáneo en el mismo medio compartido
Simplex	Solo un sentido (ej. broadcast TV)
TDD	Separa sentidos en el tiempo
FDD	Separa sentidos en frecuencia

- Ethernet moderno PC–**switch**: enlace **full duplex** → sin colisiones CSMA/CD clásicas en ese enlace.
- **Full duplex en cada tramo del anillo del lab** no evita **colas** si hay congestión.
- En el modelo, **Lnk** recibe por **toNet** o **toOut** según el sentido del flujo — coherente con enlaces bidireccionales entre vecinos.

No confundir

- Full duplex del **medio** ≠ ausencia de **congestión** en **net/Lnk**.
- Half duplex ≠ simplex.
- Colisiones en half-duplex **compartido** (hub, Wi-Fi clásico) vs enlace dedicado full duplex.

7. Bloque E — Capa de red / Tanenbaum

Referencia: A. S. Tanenbaum & D. J. Wetherall, *Computer Networks*, 5.^a ed., Prentice Hall, 2011 (capítulo de capa de red y enrutamiento).

Qué debe saber

Conceptos base

- **Routing:** decidir caminos / construir tablas.
- **Forwarding:** reenviar según **tabla** ya calculada (lo que hace **Net** en cada hop).
- **Capa de red:** entrega **extremo a extremo** a través de varias redes.
- **Datagrama (IP):** cada paquete enrutado **independientemente**; ante fallos se puede reconfigurar.
- **Circuito virtual:** fase de establecimiento + **ID de VC** en paquetes.

Algoritmos y protocolos (nivel conceptual)

Tema	Idea para el examen
Distance Vector (RIP)	Vectores de distancia a vecinos ; Bellman-Ford; problema count-to-infinity
Link State (OSPF)	Flooding de estado de enlaces; Dijkstra local; áreas para escalar
Flooding	Reenvío amplio con controles (TTL, secuencia)
TTL	Limita vida del paquete; evita loops infinitos
Bucles de routing	Paquetes circulan hasta TTL/recursos agotados
Shortest path	Minimizar métrica (saltos, costo, demora)
Routing estático	Tablas manuales
Routing dinámico	Adaptación a cambios
Default gateway	Ruta por defecto
Longest prefix match	Prefijo más específico en tabla

Relación con el Lab 4

- Decisión en **Net** = **routing/forwarding** del modelo (no TCP, no física).
- **hopCount** en **packet.msg** ↔ métrica **saltos** del enunciado.
- Baseline fijo **toLnk[0]** ↔ routing **no óptimo** (destino puede estar más cerca por el otro sentido).

- Su algoritmo debe evitar **bucles** (TTL, contadores, tablas, split horizon, etc. — lo que hayan implementado).
- **Echo/hello** ↔ mensajes de **control** (distinto del tráfico de datos), como RIP/OSPF en la realidad.
- **Prohibido cTopology**: en la realidad tampoco se “escanea el simulador”; se **aprende** la topología con mensajes de control.

Preguntas de autoevaluación

1. ¿En qué se diferencia routing de forwarding?
2. ¿Por qué RIP tiene límite práctico de 15 saltos?
3. ¿Qué problema tiene Distance Vector tras un fallo?
4. ¿Cómo se relaciona TTL con bucles en su algoritmo?

8. Bloque F — OMNeT++ / Varga & Hornig

Referencia: A. Varga & R. Hornig, “OMNeT++”, *Proc. Int. Conf. Simulation Tools and Techniques (Simutools)*, 2008 — conceptos de simulación por eventos discretos.

Qué debe saber

Concepto	Uso en el kickstarter
cSimpleModule	<code>App</code> , <code>Net</code> , <code>Lnk</code> — lógica en <code>handleMessage</code>
Módulo compuesto	<code>Node</code> contiene <code>app</code> , <code>net</code> , <code>lnk[]</code>
handleMessage	Procesa mensajes y eventos
scheduleAt(t, msg)	Programa evento en tiempo <code>t</code>
simTime()	Tiempo de simulación actual
Gates NED	<code>toNet</code> , <code>toApp</code> , <code>toLnk</code> , <code>toOut</code> , <code>toNod</code>
omnetpp.ini	Parámetros sin recompilar (<code>exponential(1)</code> , etc.)
recordScalar	Escalares al final (<code>.sca</code>)
cOutVector	Series temporales (<code>.vec</code>)
Simulación por eventos discretos	No es simulación continua en tiempo real

Concepto	Uso en el kickstarter
Canales NED	<code>datarate</code> , <code>delay</code> entre nodos
Define_Module	Registra el tipo de módulo C++
getParentModule()->getIndex()	Índice del <code>node[i]</code>
arrivedOn("toNet\$i")	Identifica gate de llegada en <code>Lnk</code>
cancelAndDelete	Limpieza de eventos en destructor
packet.msg → opp_msgc	Generar <code>packet_m.h</code> antes de compilar
finish()	Al final de la simulación
getCreationTime()	Demora desde creación del mensaje

Qué debe poder explicar

- Por qué `App` usa un **self-message** (`sendMsgEvent`) y no solo `handleMessage` espontáneo.
 - Diferencia entre **módulo** y **canal**.
 - Dónde mirar resultados: `results/*.sca, *.vec`, IDE → Analyze.
-

9. Bloque G — Paper y poster

Qué debe saber

- **Paper:** 6–8 páginas, plantilla LaTeX de la cátedra; abstract en inglés; cuerpo con experimentos y **comparación con baseline**.
- **Contenido obligatorio:** Casos 1 y 2, algoritmo propio, decisiones de diseño, métricas, ¿bucles?, mejoras cuantificadas (% o valores).
- **Métricas en el informe:**
 - **Demora** E2E (experiencia de entrega).
 - **Salto** (`hopCount` si lo instrumentaron).
 - **Buffers** (ocupación de cola en `Lnk`).
 - **Utilización de enlace** (tiempo ocupado transmitiendo).
- **Caso 2 en paper:** curva o tabla `interArrivalTime` vs demora/colas; umbral de **estabilidad** justificado.
- **Poster:** resumen visual para **muestra presencial**; objetivo, método, resultados, conclusiones; defensa oral con datos **del grupo**.

- **Votación:** 3 mejores posters con premios (no reemplaza el paper ni el código).
-

10. Respuestas que debe dominar

Estas ideas cubren muchas preguntas de las que entran en la defensa:

1. Baseline reenvía por **toLnk[0]** (horario).
 2. Destino local → **toApp**.
 3. **8 nodos, 2 interfaces**, anillo cerrado 7-0.
 4. Caso 1: **0 y 2 → 5**; Caso 2: **0,1,2,3,4,6,7 → 5**.
 5. Caso 2: umbral de **interArrivalTime** para estabilidad.
 6. Métricas: **demora, saltos, buffers, enlaces**.
 7. **No cTopology / cGate** para vecinos; **sí echo/hello**.
 8. **interArrivalTime = 0** → **sin tráfico** en App.
 9. Canales: **1 Mbps, 100 µs** delay; simulación **200 s**.
 10. **packet.msg**: **source, destination, hopCount**.
 11. Anillo: cada nodo, **dos** vecinos; baseline = **un sentido**.
 12. Full duplex: **simultáneo** en el enlace; no elimina congestión sola.
 13. Routing ≠ forwarding (Tanenbaum).
 14. RIP = **distance vector**; OSPF = **link state**.
 15. TTL / controles evitan **loops infinitos**.
 16. OMNeT++ = eventos discretos; **scheduleAt, simTime()**.
 17. Paper **6-8** páginas; poster en **muestra presencial**.
 18. Comparar siempre con **baseline de la cátedra**.
 19. **Lnk**: cola + **endServiceEvent** = servidor de enlace.
 20. **networkStar**: **57** nodos, **4** interfaces (prueba generalización).
 21. Token Ring físico ≠ anillo del lab (a menudo **estrella** + MAU).
 22. Distance Vector: **count-to-infinity**.
 23. Link State: **Dijkstra** con mapa de enlaces.
 24. Demora en App: **simTime() - getCreationTime()**.
 25. Debe discutir **bucles** si el algoritmo reenvía sin control.
-

11. Referencias bibliográficas

1. Enunciado: [2026 Lab4_enunciado.pdf](#)
2. Código: [lab4-kickstarter/](#)
3. Plantillas: [paper-template/](#) o [Paper+Poster/paper/](#), [poster-template/](#) o [Paper+Poster/poster/](#)
4. [Red en anillo — Wikipedia](#)

5. [Dúplex \(telecomunicaciones\) — Wikipedia](#)
 6. Tanenbaum & Wetherall, *Computer Networks*, 5th ed., Prentice Hall, 2011(capítulo de capa de red enrutamiento)
 7. Varga & Hornig, “OMNeT++”, Simutools 2008 (funciones que utilizamos para resolver el lab y las simulaciones)
-

12. El día de la defensa

1. Antes del cuestionario: respirar y repasar esta guía.
2. Leer cada pregunta **completa**; los distractores suelen ser valores del kickstarter mal aplicados (ej. `toLnk[1]`, `cTopology`, todos los nodos emitiendo).
3. Si no está seguro: descartar opciones absurdas (ejemplo: poster reemplaza código, RIP en `Lnk`, etc.).